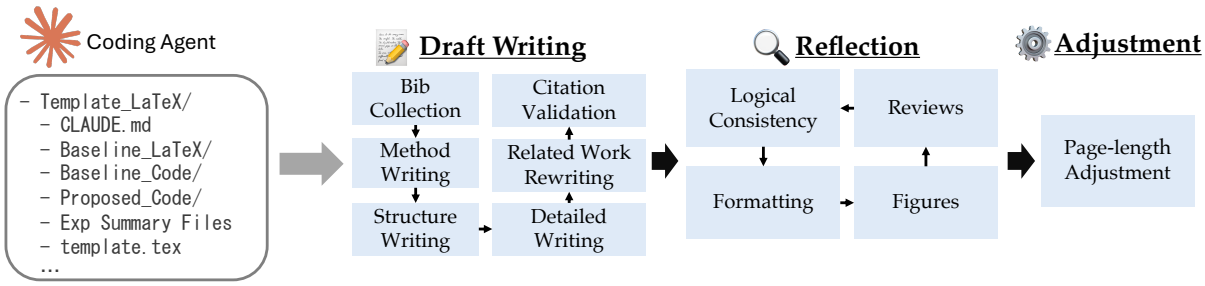

Figure 3: **Jr. AI Scientist Workflow for the Writing Phase.** The Writing process consists of three steps: Draft Writing, Reflection, and Adjustment.

the system selects the next nodes, either by initializing new nodes from the baseline code or by debugging previously generated buggy codebases. When debugging buggy codebases, we provide the coding agent with detailed runtime feedback, such as standard output and error messages, to guide iterative debugging until the issue is resolved. We set this stage to run for 12 iterations.

3.3.3 Stage2: Iterative Improvement

Stage 2 focuses on iteratively improving the method implemented in Stage 1 until its performance metrics surpass those of the baseline. In each trial, the coding agent first proposes an improvement idea to the experimental code, and then applies the modification based on the improved idea. To avoid overwriting the Stage 1 results, we instruct the coding agent to use a new entry file named `improved_proposed_method.py` as the implementation target. The system then executes this script, followed by `plot.py`, to generate results and visualizations in the same manner as Stage 1.

As shown in Figure 2, for each trial, the experimental codebase is selected probabilistically from either (1) the Stage 1 implementation or (2) the node containing the best-performing implementation observed so far. Stage 2 ends when a bug-free implementation surpasses the baseline performance. The resulting code is then passed to Stage 3. We set this stage to run for 50 iterations.

3.3.4 Stage3: Ablation Study

Stage 3 performs ablation studies on the improved method implemented in Stage 2. In each trial, the system uses an LLM to generate ablation study ideas and then employs the coding agent to implement corresponding scripts based on those ideas. To encourage higher-quality ablation ideas, we first instruct the coding agent to produce a textual description of the Stage 2 method, which is then provided to the LLM as context for generating more meaningful ablation ideas. The generated ablation ideas include hyperparameter ablations, which analyze the sensitivity of the method to key hyperparameters, and component-level ablations, which assess the contribution of each component to the overall performance. To avoid overwriting Stage 2's code, we instruct the coding agent to use new entry files named `hyperparam_ablation_study.py` and `component_ablation_study.py` as the implementation target. The iterations are executed until a sufficient number of experimental results are obtained.

3.4 Writing Phase

We primarily used a coding agent (e.g. Claude Code (Anthropic, 2025)) as a writing agent for the writing process. As shown in Figure 3, the Writing Phase consists of three stages—Draft Writing, Reflection, and Adjustment. Below, we describe the resources provided to the writing agent and the details of each stage.

3.4.1 Preliminary: Resources Provided to Writing Agent

Conference LaTeX Template. We provide the conference LaTeX template to the writing agent. We give the Agents4Science template, and the corresponding directory is set as the working directory where the writing agent operates.

Instruction Markdown File for the Writing Agent. An instruction file in Markdown format is provided to the writing agent. This document defines the overall structure of the paper, outlining how each section should be organized, and offers detailed guidelines on the key points and considerations for writing each part of the manuscript.

Baseline LaTeX Files and Code. We provide the writing agent with the baseline LaTeX files and code. These resources are mainly used to explain the baseline method in the Method section.

Stage 2 Proposed Method Code. The writing agent is also given the Stage 2 code (the proposed method). This is primarily referenced in the Method section when explaining the proposed approach.

Experiment Summaries for Each Stage. Following the protocol of AI Scientist v2, we provide the writing agent with summarized JSON files containing key experimental results for each stage (`baseline_summary.json`, `improved_research_summary.json`, `component_ablation_summary.json`, and `hyperparam_ablation_summary.json`). These files include essential information such as experimental descriptions, results, and paths for visualization results, which are crucial for the writing agent when writing the experimental results section. In addition, for ablation studies, we not only provide the JSON files but also automatically convert them into LaTeX table files (`component_ablation_summary_table.tex` and `hyperparam_ablation_summary_table.tex`). This conversion has significantly reduced numerical transcription errors in the paper.

The only materials that humans need to prepare for each paper generation are the LaTeX source files and the baseline codebase of the baseline paper. The CLAUDE.md file and the conference LaTeX template are shared across all experiments. The code and experimental files for the proposed methods are automatically generated during the experimental stage. Using these inputs, the AI agent automatically writes all sections and captions in the paper.

3.4.2 Draft Writing

As shown in Figure 3, the Draft Writing stage follows a multi-step process: it begins with the collection of BibTeX entries, followed by the writing of the Method section, the generation of the paper structure, and finally the full-paper writing. Afterward, the system performs a rewrite of the Related Work section and subsequently validates the correctness of citations. Here, we first explain how we determined the writing order and then describe the detailed procedures for each stage of this workflow.

Rationale of Writing Orders. Following AI Scientist v2 (Yamada et al., 2025), we initially generated the entire paper at once, but this resulted in a decline in the quality of the Method section. We therefore adopted a step-by-step writing process to improve overall consistency and quality. When determining the writing order, we considered it most important for the writing agent to first accurately understand and describe the proposed method, as this understanding serves as the foundation for correctly writing other sections. Therefore, we instruct the writing agent to focus exclusively on accurately writing the Method section. In addition, for the paper structure, we followed the approach of AI-Researcher (Tang et al., 2025) and introduced an intermediate step of summarizing the paper structure, in which the writing agent briefly outlines the content of each section before full-paper generation. These refinements made it possible to produce more consistent and accurate descriptions throughout a paper.

Collection of BibTeX Entries. To ensure accurate citation, it is essential to collect a complete and correct set of BibTeX entries. Following AI Scientist v1 (Lu et al., 2024), we use the Semantic Scholar API to retrieve BibTeX records. However, this approach alone often yields an insufficient number of references. To address this limitation, we adopt a practical strategy commonly used by human researchers: using the baseline paper's BibTeX file as a starting point. This approach allows the system to gather a sufficient number of references while also can expect correct citation by referring to the baseline's LaTeX source. Since the baseline's BibTeX file does not include the entry for the baseline reference set, we explicitly add it to the reference set.

Method Section Writing. For the Method section, we instruct the writing agent to write both a preview of the baseline method and a detailed description of the proposed method. To ensure an accurate description, we refer the writing agent to the LaTeX source of the baseline paper so that it can correctly describe the existing method. We also instruct the writing agent to describe the proposed method based on the Stage

2 implementation code, ensuring that the technical details are precisely reflected in the text. This process yielded more accurate and consistent Method sections.

Related Work Section Rewriting. To clearly define the position and novelty of the generated research, we instruct the writing agent to rewrite the Related Work section after completing the full paper draft. Since Jr. AI Scientist aims to update the baseline paper, the Related Work section of the baseline serves as a valuable summary of the research field and provides useful guidance on writing style and structure. Therefore, we instruct the writing agent to refer to the Related Work section in the baseline's LaTeX file when generating its own version.

Citation Validation. We introduce a citation verification phase at the end of the Draft Writing stage. Since Jr. AI Scientist updates the baseline paper, it can correctly reuse many of the original citations from the baseline paper. In this step, the writing agent compares the generated paper with the baseline LaTeX file in terms of the quantity and quality of citations, and is instructed to add missing references and remove inappropriate ones to ensure accurate and consistent citation practices.

3.4.3 Reflection

To improve the overall quality of the generated paper, we incorporated multiple reflection processes into our workflow. We repeat these reflections three times.

(1) Feedback Generation and Reflection on Logical Consistency. This process aims to enhance the academic reliability of the generated text by producing specific and actionable feedback. In particular, this process examines several key aspects essential for ensuring the logical soundness of a paper, such as logical consistency, validity of supporting citations, and alignment between experimental results and textual descriptions, and whether each section contains a sufficient amount of content. We first instruct the writing agent to generate feedback regarding the above aspects, and then use it to revise the draft based on that feedback. This process encourages the generation of more logically coherent and trustworthy papers.

(2) Reflection on Formatting and Presentation. Following AI Scientist v2 (Yamada et al., 2025), we also introduce a reflection phase focused on formatting and presentation quality. In this phase, the writing agent generates feedback such as: "Are there any LaTeX syntax errors or style violations we can fix? Refer to the chktex output below", or "Are there short sections (one or two sentences) that could be combined into a single paragraph?" This process helps produce a final draft that is well-formatted and stylistically consistent.

(3) Feedback Generation and Reflection on Figures. Following AI Scientist v2 (Yamada et al., 2025), we perform figure-level reflection in the refinement stages by integrating a Large Multimodal Model (LMM)-based feedback mechanism. This process aims to improve the quality, clarity, and alignment of generated figures, captions, and their corresponding textual interpretations. Specifically, the LMM is used to identify figures that are uninformative or make little contribution to the paper's scientific value, and such figures are either removed or moved to the Appendix. This ensures that all figures presented in the main paper contain adequate informational value. To achieve this, we provide the LMM with the paper's abstract, figure captions, and figure images to generate targeted feedback, which is then used to guide the reflection and revision process.

(4) Feedback Generation and Reflection from AI Reviews. Following CycleResearcher (Weng et al., 2025a), we adopt a review-based reflection, where the system improves its manuscript based on reviewer feedback. In this step, the writing agent revises the generated paper according to reviewer comments such as "the Method section is unclear", "important parameter details are missing", or "the writing is overly verbose". For generating such feedback, we employ AI reviewers in AI Scientist v1 (text-only evaluation) and v2 (evaluation including figures). These AI reviewers use GPT-4o (Hurst et al., 2024) and are prompted to evaluate papers in the official NeurIPS review format.

3.4.4 Adjustment: Page-length Adjustment

We also introduced a new design to the page-length adjustment process. In AI Scientist v2 (Yamada et al., 2025), when the generated paper exceeded the predefined page limit, the system attempted to adjust the length within a single LLM call. However, this approach often resulted in over-trimming, causing the paper